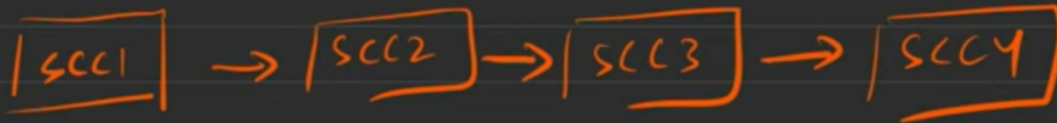


2.10

Strongly Connected Components $\rightarrow$ Kosaraju's Algorithm.



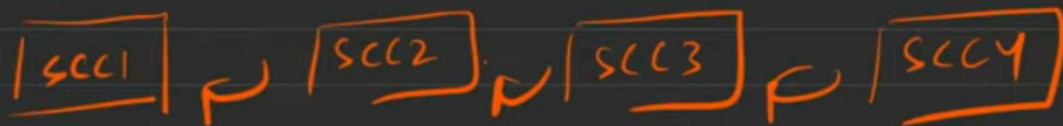
Thought Process



Strongly Connected Components $\rightarrow$ Kosaraju's Algorithm.



Thought Process



Strongly Connected Components $\rightarrow$ Kosaraju's Algorithm.



2.10

DFS(0)

DFS(1)

DFS(2)

DFS(3)

DFS(4)

DFS(7)

DFS(5)

DFS(6)

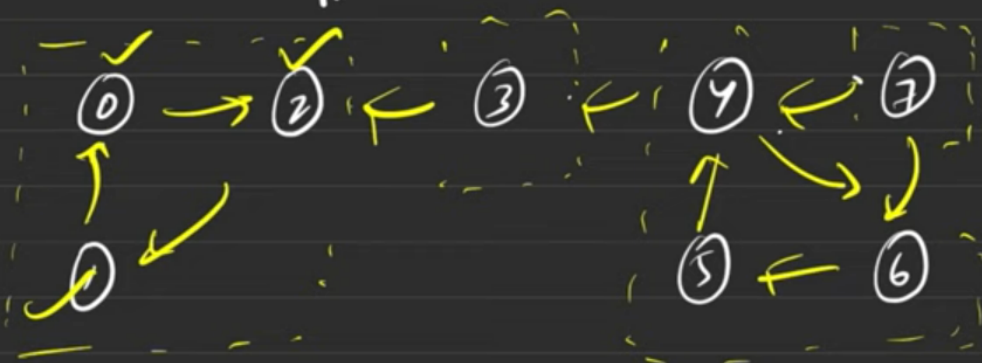
(.) Do a dfs

top →

0
1
2
3
4
5
6
7

→

Strongly Connected Components $\rightarrow$ Kosaraju's Algorithm.



$\hookrightarrow$ sort all the edges acc to finishing time

$\hookrightarrow$ Reverse the Graph

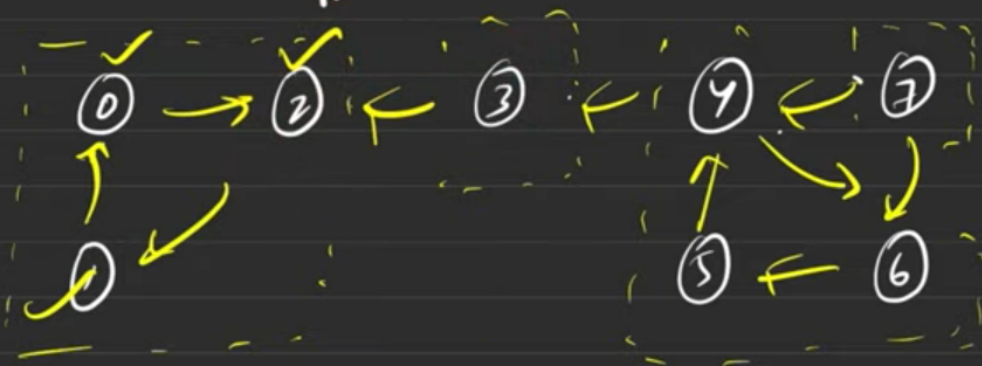
$\hookrightarrow$ Do a dfs

DFS(0)
 DFS(2)
 DFS(1)

top $\rightarrow$ 0
 1
 2
 3
 4
 5

2.10

Strongly Connected Components $\rightarrow$ Kosaraju's Algorithm.



DFS(3)

0 2 1
3

$\hookrightarrow$ sort all the edges acc to finishing time

Reverse the Graph

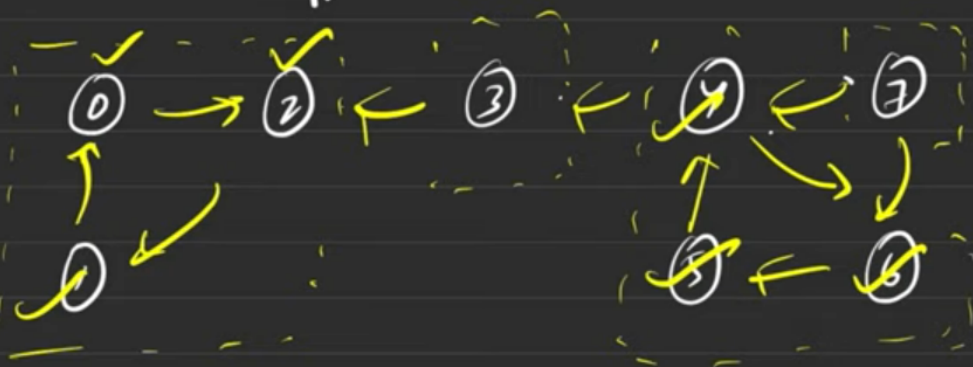
Do a dfs

top $\rightarrow$ 0
1
2
3
4
5

G-54. Strongly Connected Components - Kosaraju's Algorithm

2.10

Strongly Connected Components $\rightarrow$ Kosaraju's Algorithm.



0 2 1
3
4 6 5

DPS(4)
↓
DPS(6)
↓
DPS(5)

Sort all the edges acc to finishing time

Reverse the Graph

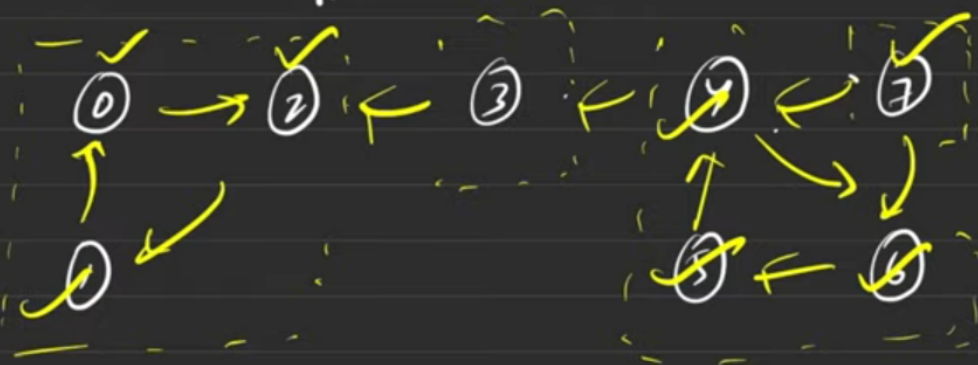
Do a dfs

top $\rightarrow$ 0
1
2
3
4
5
6



2.10

Strongly Connected Components $\rightarrow$ Kosaraju's Algorithm.



DFS(7)

0 2 1
3
4 6 5
7

$\rightarrow$ sort all the edges acc to finishing time

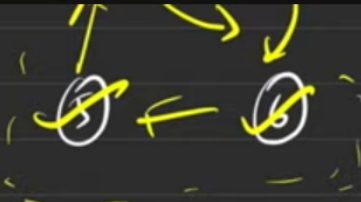
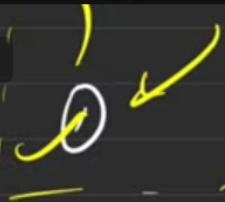
Reverse the Graph

Do a dfs

top $\rightarrow$ 0
1
2
3
4
5
6
7

G-54. Strongly Connected Components - Kosaraju's Algorithm

2.10



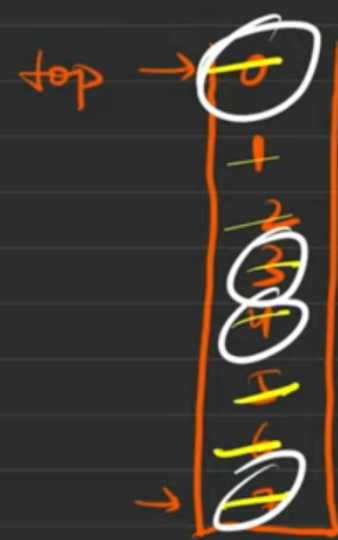
edges are so finishing time

✓ Reverse the Graph

(*) Do a dfs

0 2 1
3
4 6 5
7

Dfs() → 7 nodes



scc1

scc2

scc3

scc4

G-54. Strongly Connected Components - Kosaraju's Algorithm

Problems Courses Get Hired Contests POTD

Practice

```
47: 2.10 Solution
48: {
49:     private void dfs(int node, int[] vis, ArrayList<ArrayList<Integer>> adj,
50:         Stack<Integer> st) {
51:         vis[node] = 1;
52:         for(Integer it: adj.get(node)) {
53:             if(vis[it] == 0) {
54:                 dfs(it, vis, adj, st);
55:             }
56:         }
57:         st.push(node);
58:     }
59:     private void dfs3(int node, int[] vis, ArrayList<ArrayList<Integer>> adjT) {
60:         vis[node] = 1;
61:         for(Integer it: adjT.get(node)) {
62:             if(vis[it] == 0) {
63:                 dfs3(it, vis, adjT);
64:             }
65:         }
66:     }
67:     //Function to find number of strongly connected components in the graph.
68:     public int kosaraju(int V, ArrayList<ArrayList<Integer>> adj)
69:     {
70:         int[] vis = new int[V];
71:         Stack<Integer> st = new Stack<Integer>();
72:         for(int i = 0; i < V; i++) {
73:             if(vis[i] == 0) {
74:                 dfs(i, vis, adj, st);
75:             }
76:         }
77:
78:         ArrayList<ArrayList<Integer>> adjT = new ArrayList<ArrayList<Integer>>();
79:         for(int i = 0; i < V; i++) {
80:             adjT.add(new ArrayList<Integer>());
81:         }
82:         for(int i = 0; i < V; i++) {
83:             vis[i] = 0;
84:             for(Integer it : adj.get(i)) {
85:                 // i -> it
86:                 // it -> i
87:                 adjT.get(it).add(i);
88:             }
89:         }
90:         int scc = 0;
91:         while(!st.isEmpty()) {
92:             int node = st.peek();
93:             st.pop();
94:             if(vis[node] == 0) {
95:                 scc++;
96:                 dfs3(node, vis, adjT);
97:             }
98:         }
99:         return scc;
100:     }
101: }
```

C++ (g++ 5.4)

Average Time: 20m

```
1: // } Driver Code Ends
2:
3:
4: class Solution
5: {
6: private:
7:     void dfs(int node, vector<int> &vis, vector<int> adj[],
8:         stack<int> &st) {
9:         vis[node] = 1;
10:         for(auto it: adj[node]) {
11:             if(!vis[it]) {
12:                 dfs(it, vis, adj, st);
13:             }
14:         }
15:         st.push(node);
16:     }
17: private:
18:     void dfs3(int node, vector<int> &vis, vector<int> adjT[]) {
19:         vis[node] = 1;
20:         for(auto it: adjT[node]) {
21:             if(!vis[it]) {
22:                 dfs3(it, vis, adjT);
23:             }
24:         }
25:     }
26: public:
27:     //Function to find number of strongly connected components in the graph.
28:     int kosaraju(int V, vector<int> adj[])
29:     {
30:         vector<int> &vis(V, 0);
31:         stack<int> st;
32:         for(int i = 0; i < V; i++) {
33:             if(!vis[i]) {
34:                 dfs(i, vis, adj, st);
35:             }
36:         }
37:     }
38: }
```



G-54. Strongly Connected Components - Kosaraju's Algorithm



Problems
2.10

Courses Get Hired Contests POTD

Problem Editorial Submissions Comments

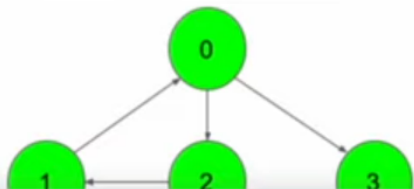
C++ (g++ 5.4)

Average Time: 20m

Given a Directed Graph with V vertices (Numbered from 0 to $V-1$) and E edges, Find the number of strongly connected components in the graph.

Example 1:

Input:



Output Window



Compilation Results Custom Input

Problem Solved Successfully

You get marks only for the first correct submission if you solve the problem without viewing the full solution.

Test Cases Passed:

161 / 161

Your Total Score:

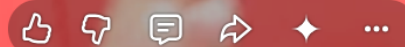
507

Total Time Taken:

Correct Submission

Count:

```
32 //Function to find number of strongly connected components in the graph.
33 int kosaraju(int V, vector<int> adj[])
34 {
35     // 0(2xV)
36     vector<int> vis(V, 0);
37     stack<int> st;
38     // 0(V+E)
39     for(int i = 0; i < V; i++) {
40         if(!vis[i]) {
41             dfs(i, vis, adj, st);
42         }
43     }
44     // 0(V + E)
45     vector<int> adjT[V];
46     // 0(V+E)
47     for(int i = 0; i < V; i++) {
48         vis[i] = 0;
49         for(auto it : adj[i]) {
50             // i -> it
51             // it -> i
52             adjT[it].push_back(i);
53         }
54     }
55     int scc = 0;
56     // 0(V+E)
57     while(!st.empty()) {
58         int node = st.top();
59         st.pop();
60         if(!vis[node]) {
61             scc++;
62             dfs3(node, vis, adjT);
63         }
64     }
65     return scc;
66 }
67
68
69
```



22:10 / 22:43

Complexity and conclusion >

